



SECURE CODING REVIEW

CodeAlpha Cyber
Security Internship

PREPARED BY: Muhammad
Awais

TASK 2: Secure Coding Review

INTRODUCTION

Secure Coding is the practice of writing software in a way that protects applications from security vulnerabilities and cyber attacks.

Many Security breaches occur because developers unintentionally introduce weaknesses into their code

The purpose of this code is to identify security flaws in a simple application and provide recommendations to make the code more secure.





PROJECT OBJECTIVES

- Identify Secure vulnerabilities in a source code
- Analyze the risks associated with insecure coding.
- Implement secure coding techniques
- Improve Application security
- Learn best practices for secure software development.

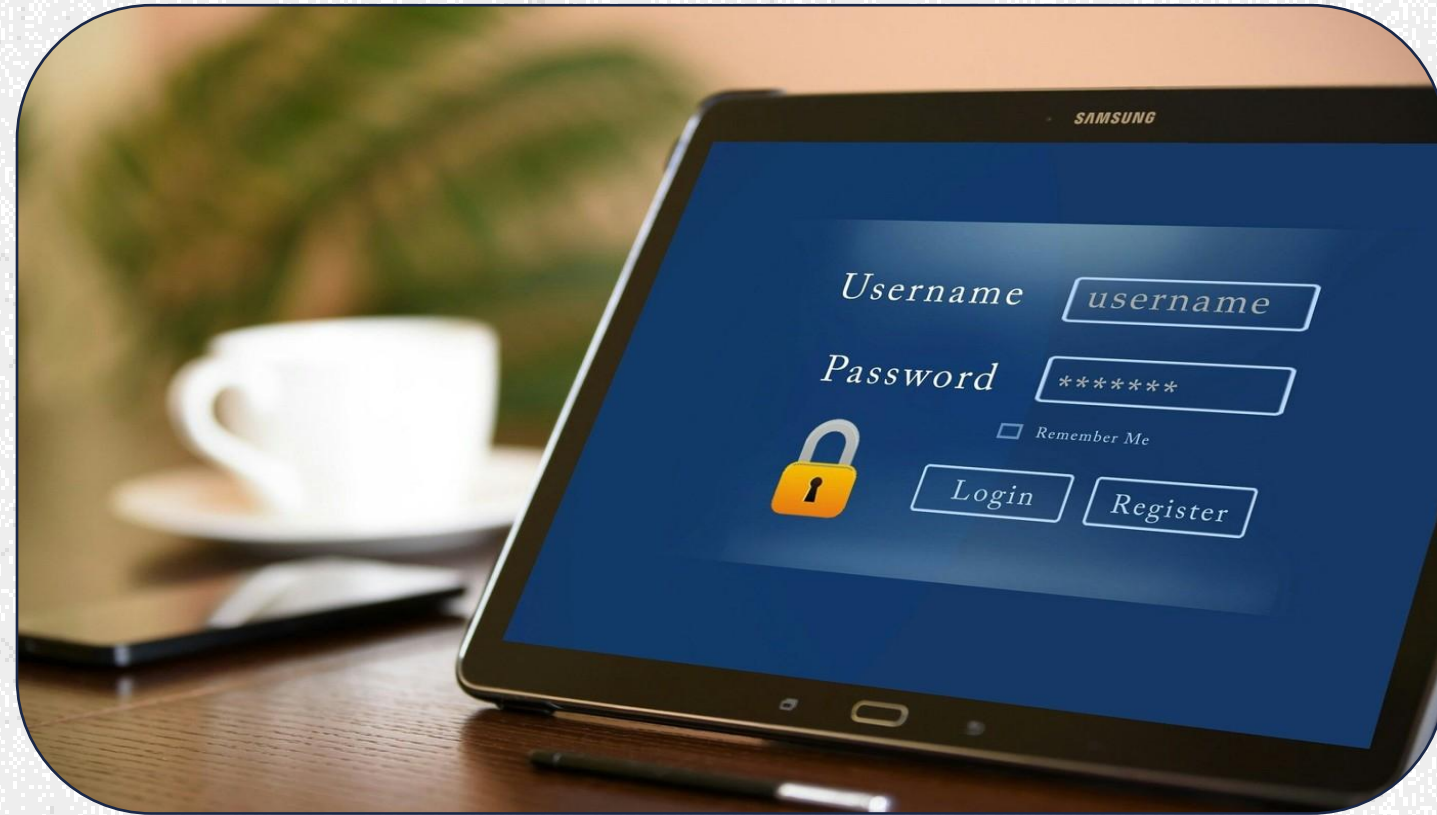
VULNERABLE LOGIN SYSTEM

The following Login System Contains Multiple security weaknesses.

Features:

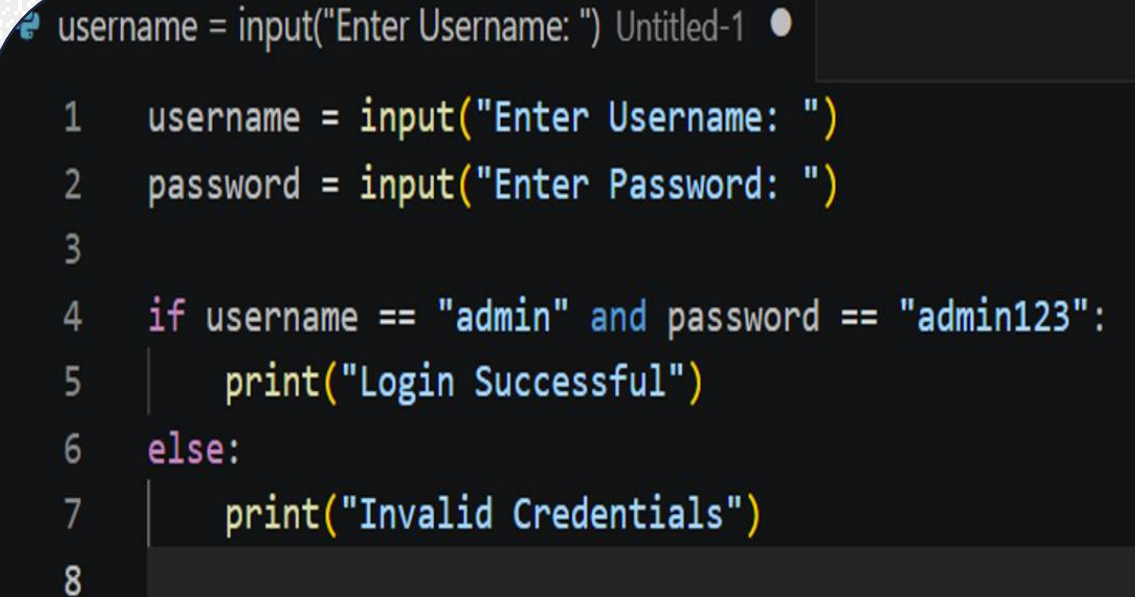
- Accept username and password.
- Check credentials directly in code
- Provide access when credentials match

However, the implementation is insecure and can expose sensitive information.



Vulnerable Code

```
username = input("Enter  
Username: ")  
password = input("Enter  
Password: ")  
if username == "admin" and  
password == "admin123":  
    print("Login  
Successful")  
else:  
    print("Invalid  
Credentials")
```

A screenshot of a code editor window titled "Untitled-1". The code is written in Python and is identical to the code shown in the first block. The editor has a dark background with light-colored text. The code is as follows:

```
username = input("Enter Username: ")  
1 username = input("Enter Username: ")  
2 password = input("Enter Password: ")  
3  
4 if username == "admin" and password == "admin123":  
5     print("Login Successful")  
6 else:  
7     print("Invalid Credentials")  
8
```

VULNERABILITY 1- HARDCODED CREDENTIALS

- Description:

- The username and password are directly written within inside the source code.

- Example:

Username: admin

Password: admin123

Risks:

- Anyone will access to the code can see the credentials.
- Attackers can easily gain unauthorized access.
- Credentials become difficult to manage and update

Severity:

High

VULNERABILITY 2- PLAIN TEXT PASSWORD STORAGE

- **Description:**

Password are stored and compared as plain text

- **Severity:**

Medium

Risks:

- Password can be exposed if the source code is leaked.
- Attackers can read passwords directly.
- No protection is applied to sensitive data.

VULNERABILITY 3 – LACK OF INPUT VALIDATION

- **Description:**

The application accepts user input without validation.

- **Severity:**

High

Risks:

- Unexpected input may cause application errors.
- Can introduce security vulnerabilities in larger system
- Makes applications less secure and reliable

SECURE LOGIN SYSTEM

To Improve Security:

- Password hashing is implemented
- Credentials are protected
- Better Security are followed

This reduce the risk of credentials theft and unauthorized access



Secure Code

```
import hashlib
```

```
stored_username = "admin"  
stored_password =  
hashlib.sha256("admin123".encode()).hexdigest()
```

```
username = input("Enter Username: ")  
password = input("Enter Password: ")
```

```
hashed_password =  
hashlib.sha256(password.encode()).hexdigest()
```

```
if username == stored_username and  
hashed_password == stored_password:  
    print("Login Successful")  
else:  
    print("Invalid Credentials")
```

```
Secure_code  
1  import hashlib  
2  
3  stored_username = "admin"  
4  stored_password = hashlib.sha256("admin123".encode()).hexdigest()  
5  
6  username = input("Enter Username: ")  
7  password = input("Enter Password: ")  
8  
9  hashed_password = hashlib.sha256(password.encode()).hexdigest()  
10  
11 if username == stored_username and hashed_password == stored_password:  
12     print("Login Successful")  
13 else:  
14     print("Invalid Credentials")  
15
```

BENEFITS OF SECURE CODE

- **ADVANTAGES:**
- Passwords are stored in hashed form.
- Attackers cannot directly read passwords.
- Better protection of user information.
- Improved Application security
- Follow secure coding principles



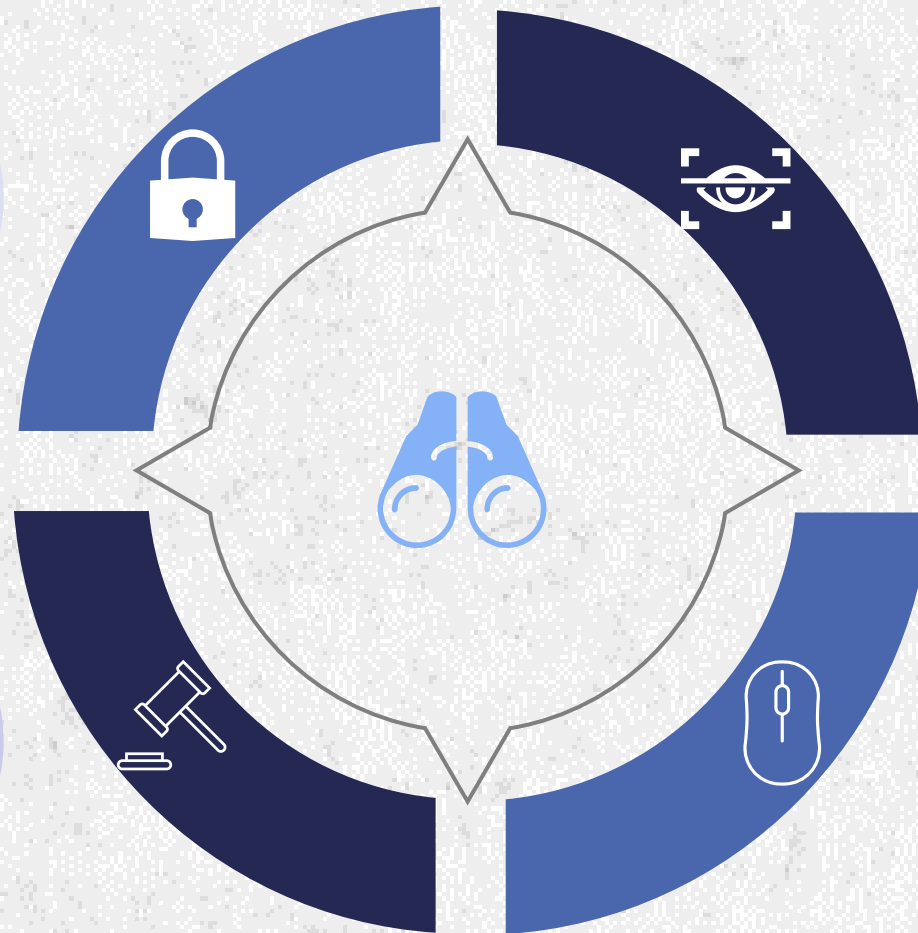
RECOMMENDATIONS

Never hardcode
password

Store passwords using
hashing algorithms

Apply the principles of
least privilege

Validate all user input



CONCLUSION

Key Takeaways

1. This Project demonstrated how insecure coding practices can introduce security vulnerabilities
2. By Identifying weaknesses and implementing secure coding techniques, applications become more resistant to cyber attacks and data breaches.
3. Secure coding is an essential part of modern cybersecurity and software development.



THANK YOU
